

Implementação e Análise de Convergência para o Cálculo de pi

Implemente um programa em C que calcule uma aproximação de pi usando uma série matemática, variando o número de iterações e medindo o tempo de execução. Compare os valores obtidos com o valor real de pi e analise como a acurácia melhora com mais processamento. Reflita sobre como esse comportamento se repete em aplicações reais que demandam resultados cada vez mais precisos, como simulações físicas e inteligência artificial.

Código:

```
#include <stdio.h>
#include <math.h>
#include <time.h>

int main(){
    long long N = 10000000000;
    long long checkpoint = 1000;

    double pi_real = 3.141592653589793;
    double pi_aprox = 0.0;
    int sinal = 1;

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    for(long long i=0; i<N; i++){
        pi_aprox += sinal * (4.0/ (2.0*i + 1.0));
        sinal *= -1;

        if(i+1 == checkpoint) {
            clock_gettime(CLOCK_MONOTONIC, &end);

            double tempo_gasto = (end.tv_sec - start.tv_sec) + (end.tv_nsec -
start.tv_nsec) / 1e9;
            double erro = fabs(pi_real - pi_aprox);

            printf("Iterações: %11lld | Pi: %.10f | Erro: %.10f | Tempo: %f s\n",
checkpoint, pi_aprox, erro, tempo_gasto);

            checkpoint *=10;
        }
    }
```

```

}
return 0;
}

```

Saída:

Iteracoes:	10000	Pi:	3.1414926536	Erro:	0.0001000000	Tempo:	0.000124 s
Iteracoes:	100000	Pi:	3.1415826536	Erro:	0.0000100000	Tempo:	0.000469 s
Iteracoes:	1000000	Pi:	3.1415916536	Erro:	0.0000010000	Tempo:	0.003279 s
Iteracoes:	10000000	Pi:	3.1415925536	Erro:	0.0000001000	Tempo:	0.029252 s
Iteracoes:	100000000	Pi:	3.1415926436	Erro:	0.0000000100	Tempo:	0.287414 s
Iteracoes:	1000000000	Pi:	3.1415926526	Erro:	0.0000000010	Tempo:	2.849834 s
Iteracoes:	10000000000	Pi:	3.1415926535	Erro:	0.0000000001	Tempo:	28.481198 s

Pi pode ser calculado por esta sequência infinita:

$$\pi = \frac{4}{1} - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \dots$$

A equação:

$(4.0 / (2.0 * i + 1.0))$

Durante o laço for:

Quando $i = 0$: O cálculo de baixo fica $(2 \times 0) + 1 = 1$. A conta inteira fica $4/1$. O sinal é positivo. Resultado: $+4.0$

Quando $i = 1$: O cálculo de baixo fica $(2 \times 1) + 1 = 3$. A conta inteira fica $4/3$. O sinal é negativo. Resultado: $-1.333...$

Quando $i = 2$: O cálculo de baixo fica $(2 \times 2) + 1 = 5$. A conta inteira fica $4/5$. O sinal é positivo. Resultado: $+0.8$

Quando $i = 3$: O cálculo de baixo fica $(2 \times 3) + 1 = 7$. A conta inteira fica $4/7$. O sinal é negativo. Resultado: -0.571

`fabs`: é uma função da biblioteca matemática do C `<math.h>` que serve para calcular o valor absoluto de números decimais.

Como ele é usado para $(\pi_{\text{real}} - \pi_{\text{aprox}})$ só preciso saber a distância entre o valor real e calculado, então preciso do valor positivo.

Analisando os resultados obtidos, fica evidente a relação direta e custosa entre o esforço computacional e o ganho de precisão. A cada vez que o número de iterações é multiplicado por 10, o erro cai na mesma proporção (ganhando-se uma casa decimal exata de acurácia), mas o tempo de execução também sofre um aumento de aproximadamente 10 vezes — saltando de frações de milissegundo no início para longos 28,4 segundos na marca de 10 bilhões de cálculos.

Esse comportamento de "retornos decrescentes" espelha exatamente o desafio de aplicações reais complexas, como simulações físicas detalhadas e o treinamento de modelos de inteligência artificial: alcançar os primeiros níveis de precisão é rápido e barato computacionalmente, mas refinar um modelo para extrair as últimas frações de acurácia exige um sacrifício drástico e quase exponencial de tempo, energia e poder de processamento.